

Universitatea din Pitești
Facultatea de Electronică și Calculatoare

Modele de procese software

Lect. univ. dr. Adrian Țurcanu
Lect. univ. dr. Cristina Tudose

Cuprins

- 1 Introducere
- 2 Modelul cascadă
- 3 Prototipizarea
- 4 Dezvoltarea incrementală
- 5 Modelul spirală
- 6 Dezvoltarea formală

Introducere

- Un program sau un sistem de programe este rezultatul unui **proces**.
- Un **model al unui proces** reprezintă o abstractizare a acestuia care include principalii pași efectuați pentru producerea sistemului.
- Descrierea procesului este una generală, pentru că nu este posibil să se definească un standard unic pentru toate tipurile de aplicații.

Modele de proces software

- Modelul cascadă
- Prototipizarea
 - prototipizarea de încercare
 - prototipizarea evolutivă
- Dezvoltare incrementală
- Modelul spirală
- Dezvoltarea formală

Modelul cascadă(waterfall model)

- Descris de Royce în 1970, modelul cascadă a devenit unul din modelele standard folosit pentru descrierea generală a procesului de dezvoltare a programelor.
- Prezintă dezvoltarea unui program ca o succesiune liniară de etape, de la analiza cerințelor și până la livrarea produsului către client și asigurarea mentenanței.
- Fiecare etapă corespunde unei activități, are un termen de finalizare și are ca scop producerea unor documente sau programe.

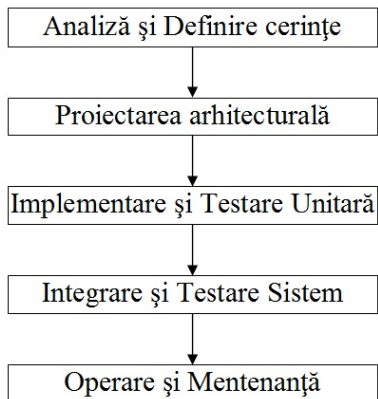
Etapele modelului cascadă

- **analiza și definirea cerințelor**: sunt stabilite serviciile, constrângerile și scopurile sistemului prin consultare cu utilizatorul
- **proiectarea arhitecturală**: se stabilește o arhitectură de ansamblu și funcțiile sistemului software pornind de la cerințe (comportamentul sistemului)
- **implementarea și testarea unitară**: designul sistemului este transformat într-o mulțime de unități de program (module); se testează dacă fiecare modul este conform cu specificația

Etapele modelului cascadă

- **integrarea și testarea sistemului**: unitățile de program sunt integrate și testate ca un sistem complet care este apoi livrat clientului
- **operare și mentenanță**: sistemul este folosit în practică; mentenanța implică corectarea erorilor sau adaugarea de noi funcționalități

Modelul cascadă



Avantaje vs. dezavantaje

- A1: Sistemul este bine documentat și bine structurat.
- A2: Permite un bun management al proiectului.
- A3: Produce sisteme robuste.
- D1: Dezvoltarea unui sistem software nu este de obicei un proces liniar ci etapele se întrepătrund.
- D2: Sunt produse multe documente până la dezvoltarea unui program care să demonstreze că sistemul funcționează.
- D3: Specificațiile nu mai pot fi modificate după ce sunt aprobate.
- D4: Erorile sunt identificate târziu.

CONCLUZIE: Modelul cascadă este unul riguros care trebuie folosit atunci când specificațiile sistemului sunt clare și nu se modifică pe parcurs.

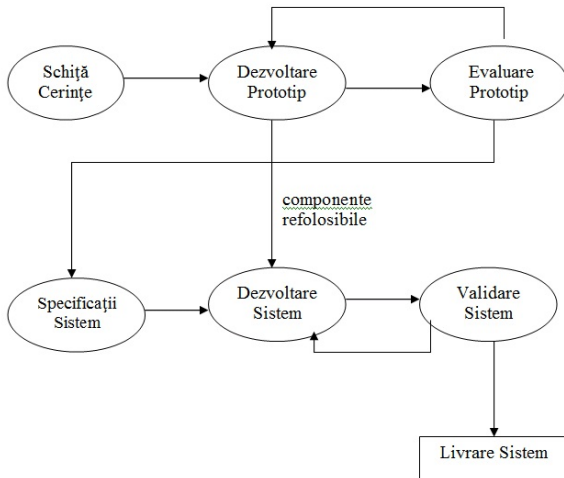
Prototipizarea

- Un **prototip** este o versiune inițială a unui sistem care este folosită pentru demonstrarea unor concepte, verificarea unor opțiuni de design și, în general, pentru a afla mai multe despre problemă și posibilele soluții.
- Prototipurile se folosesc, în special, pentru sisteme noi, complexe, pentru care nu se poate construi o specificație completă înainte de implementare și utilizare.
- Prototipurile sunt construite incremental și oferă clientului o parte din funcționalitățile viitorului sistem și o interfață.
- Există două categorii de prototipuri: **prototipuri de încercare** și **prototipuri evolutive**.

Prototipuri de încercare

- sunt prototipuri folosite pentru clarificarea cerințelor
- îndeplinesc doar o parte din cerințele clientului (pe cele mai ambigue)
- după utilizare, sunt înlăturate, iar dezvoltarea sistemului începe aproape de la 0
- au un timp redus de viață și performanțe reduse având doar scopul de a ilustra anumite cerințe

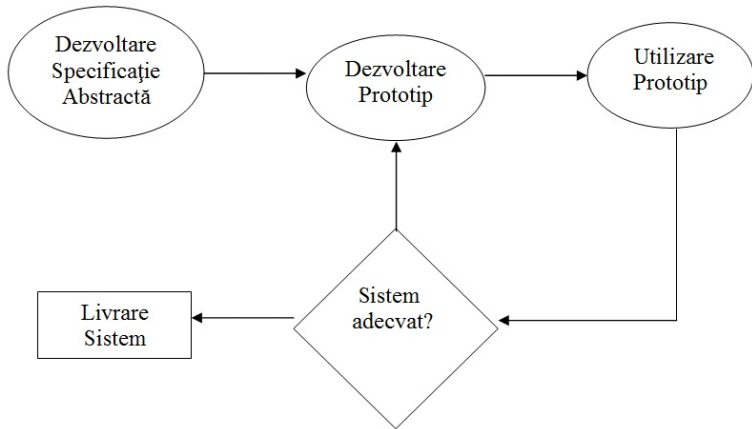
Prototipizarea de încercare



Prototipuri evolutive

- sunt folosite ca etape intermediare în dezvoltarea sistemului
- primul prototip conține cerințele cele mai clare și mai importante
- acesta este apoi rafinat succesiv până când se obține un sistem care îndeplinește toate funcționalitățile
- trebuie să fie robuste pentru că au o durată de viață mare; se construiesc respectând aceleași standarde de calitate ca pentru orice alt software.

Prototipizarea evolutivă



Probleme ale prototipizării de încercare

- lipsa unei specificații care să poată fi validată de către client
- caracteristici importante pot să nu fie incluse în prototip
- poate crea așteptări false din partea clientului privind timpul și costul dezvoltării sistemului

Probleme ale prototipizării evolutive

- prototipurile evoluează rapid și astfel nu este eficient din punct de vedere al costurilor să se creeze documentația care se creează în mod obișnuit
- modificările succesive pot afecta structura sistemului ceea ce poate genera probleme de mentenanță
- nu există o specificație riguroasă asupra căreia dezvoltatorul și clientul să cadă de acord
- accentul cade pe îndeplinirea funcționalităților fiind neglijate aspecte ca eficiența sau adaptabilitatea.

Dezvoltarea prototipurilor

- Pentru dezvoltarea de prototipuri se folosesc limbaje de nivel foarte înalt ca Smalltalk, PROLOG, LISP, SETL sau limbaje din generația a 4-a (*4GL*).
- Un limbaj *4GL* este asociat unui *SGBD* și conține: un limbaj de interogare (SQL), generator de interfețe, foi de calcul și generatoare de rapoarte.
- Principalele avantaje ale prototipizării sunt livrarea rapidă a unor părți esențiale din sistem și implicarea utilizatorului în procesul de dezvoltare și validare a acestuia.

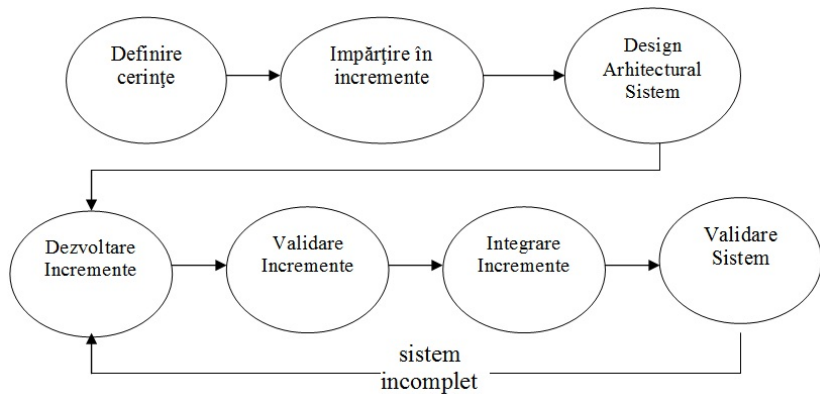
Dezvoltarea incrementală

- este o cale de mijloc între modelul cascadă și prototipizare
- are la bază următoarea idee: dacă un sistem este prea complex pentru a fi conceput sau realizat într-o singură fază atunci este mai bine ca acesta să fie dezvoltat evolutiv
- începe cu un studiu de fezabilitate al proiectului urmat de identificarea cerințelor pe care trebuie să le satisfacă sistemul
- dezvoltarea și livrarea sistemului este împărțită în **incremente**
- fiecare increment implementează o parte din funcționalități și este dezvoltat după modelul cascadă

Dezvoltarea incrementală

- **cerintele sunt ordonate după prioritate**, cele cu prioritatea cea mai mare făcând parte din primul increment, ș.a.m.d.
- scopul livrării primului increment este obținerea unei reacții din partea utilizatorului
- fiecare din următoarele incremente include analiza ultimei versiuni și adăugarea de noi funcționalități
- odată ce dezvoltarea unui increment a început, cerințele pentru acel increment nu mai pot fi modificate, dar cerințele pentru noile incremente pot fi modificate
- aceste modificări pot apărea ca urmare a analizei versiunii anterioare
- un proiect normal conține între 10 și 50 de incremente

Dezvoltarea incrementală



Avantaje ale dezvoltării incrementale

- spre deosebire de modelul cascadă în care se pregătește mai întâi toată documentația, în cazul dezvoltării incrementale, în fiecare etapă, este livrat un produs executabil, care satisface o parte din cerințele utilizatorului
- primul increment conține principalele funcționalități deci sistemul poate fi utilizat și evaluat imediat
- scade riscul de eșec deoarece fiecare increment ține cont de analiza celor anterioare
- cerințele cele mai importante fac parte din primele incremente, deci acestea vor fi testate cel mai mult. Astfel, este mai puțin probabil să apară erori în părțile cele mai importante.

Dezavantaje ale dezvoltării incrementale

- se bazează pe evoluția prototipurilor, procedeu greu de realizat, în general, datorită rigidității sistemelor
- fiecare nou increment poate duce la reorganizarea arhitecturii sistemului, acesta devenind astfel greu de verificat și întreținut
- cerințele clientului sunt greu de transformat în incremente de mărimea potrivită
- erorile de proiectare sunt greu de eliminat
- urmărind dezvoltarea sistemului clientul poate cere din ce în ce mai mult

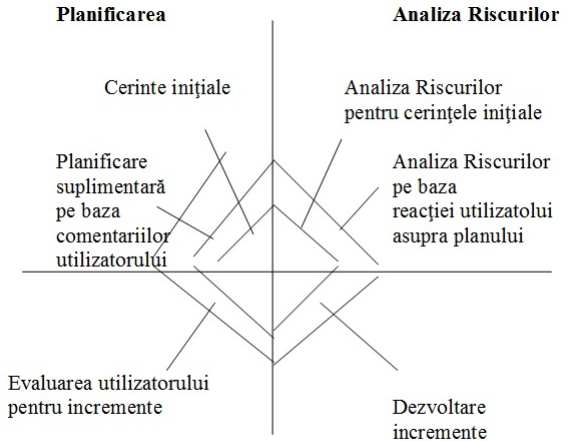
Strategii în dezvoltarea incrementală

- **Planificarea:**
 - elaborarea unui plan de acțiuni este esențială în dezvoltarea incrementală
 - presupune o analiză preliminară a proiectului în care sunt identificate principalele riscuri, se ordonează cerințele, se stabilește natura fiecărui increment și ordinea dezvoltării acestora
- **Construirea incrementelor în paralel**
 - incremente diferite pot fi construite în paralel de echipe diferite
 - cum fiecare increment are părți comune cu altele, este necesară o bună comunicare și coordonare între echipe
 - există riscul ca incrementele să nu se potrivească

Modelul spirală

- A fost definit de Barry Boehm în 1988 plecând de la punctele slabe ale modelului cascadă, în special lipsa de flexibilitate a acestuia la schimbări ale cerințelor.
- Este focalizat pe analiza riscurilor în mod incremental, repetând modelul cascada într-o serie de cicluri.
- Fiecare ciclu are 4 faze:
 - 1 **planificarea**: definirea produsului, determinarea obiectivelor și a constrângerilor
 - 2 **analiza riscurilor**: pentru cerințele inițiale și apoi pe baza reacțiilor utilizatorului la incremente
 - 3 **dezvoltarea produsului**: fiecare increment este dezvoltat, testat, integrat
 - 4 **evaluarea de către utilizator**: incrementul livrat este evaluat, iar pe baza acestei evaluări este planificat următorul ciclu

Modelul spirală



Avantaje vs. dezavantaje

- A1: este o îmbunătățire a modelului cascadă deoarece prevede mai multe livrări și mai multe posibilități de implicare a clientului.
- A2: spre deosebire de alte modele urmărește diminuarea riscurilor (de exemplu, depășirea termenului de finalizare sau a costurilor)
- D1: presupune o dezvoltare de tip incremental, deci o serie de modificări asupra sistemului
- D2: creșterea razei spiralei înseamnă creșterea costurilor

Dezvoltarea formală

- este o strategie de dezvoltare software bazată pe **transformarea unei specificații formale**(bazată pe relații matematice) într-un program executabil
- cerințele sistemului sunt exprimate prin intermediul unor specificații formale
- procesul de transformare înlocuiește etapele de proiectare arhitecturală, implementare și testare unitară din modelul cascadă
- **transformarea presupune o secvență de rafinari succesive** a căror corectitudine este demonstrată matematic
- validarea implementării se face, de asemenea, prin **demonstrații**(manuale sau automate)

Dezvoltarea formală

- Probleme specifice:
 - implică personal specializat
 - unele aspecte ale sistemului sunt dificil de specificat formal
- Utilizare: dezvoltarea de sisteme critice din punct de vedere al securității(safety-critical)

Concluzii

- Există o serie de modele de procese care pot sta la baza dezvoltării unui produs software, fiecare dintre acestea având o serie de avantaje și dezavantaje.
- Alegerea unui model depinde de natura sistemului ce urmează a fi dezvoltat.
- Parcurgerea etapelor unui model nu garantează corectitudinea sau eficiența sistemului, în fiecare dintre faze fiind posibilă apariția unor erori.
- Modelele de procese precum și programele care pot fi utilizate pentru parcurgerea anumitor etape sunt într-o permanentă dezvoltare.